NUST CHIP DESIGN CENTRE

RISC-V Architecture

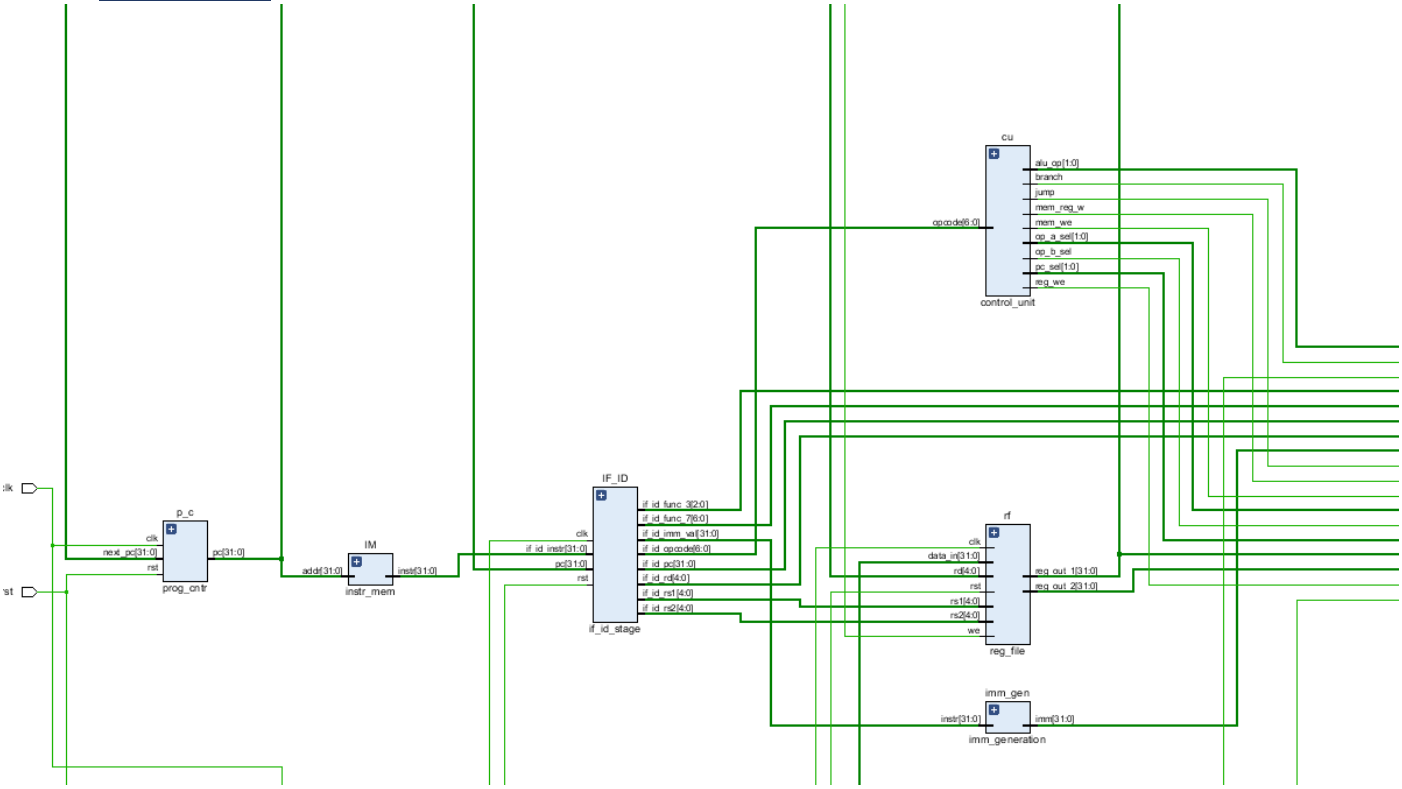
Lab Manual # 22
Handling Data Hazards

Name	*Muddassir Ali Siddiqui*
Instructor	*Sir Imran & Sir Umer*
Date	*3rd Sep 2025*

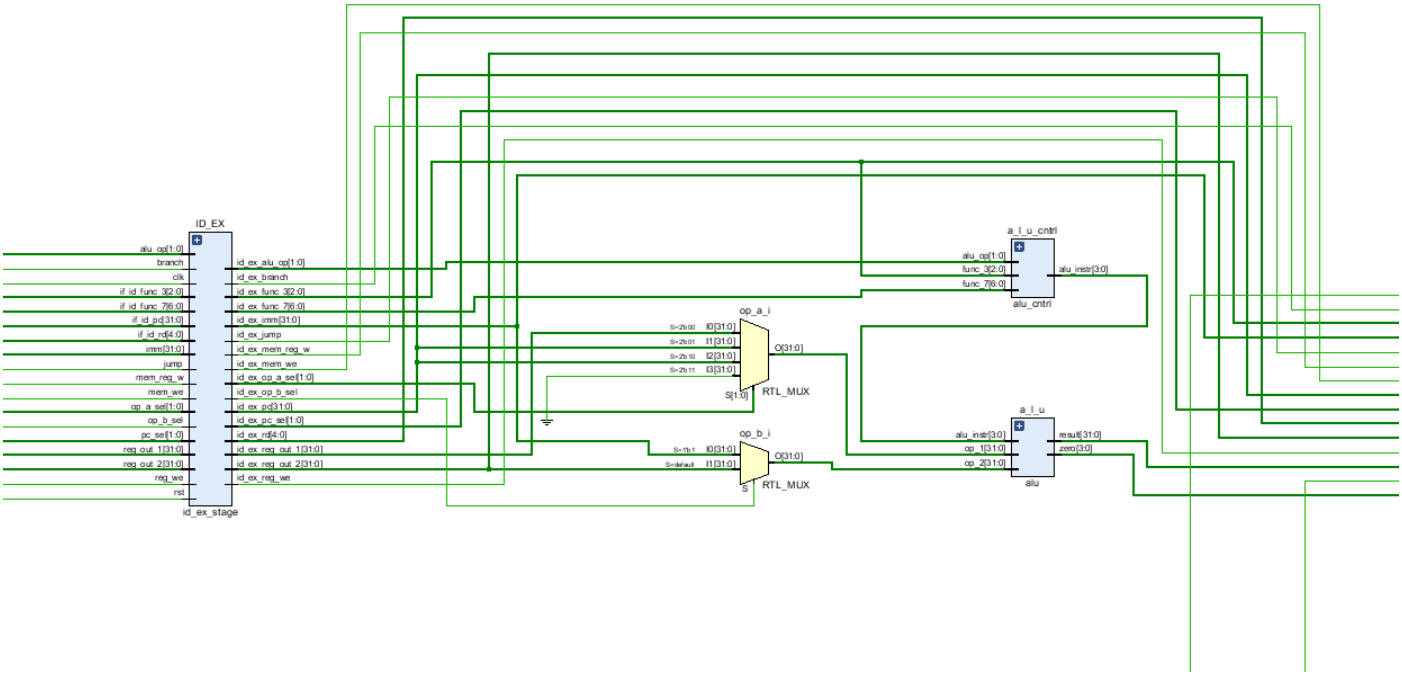
1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Schematics:

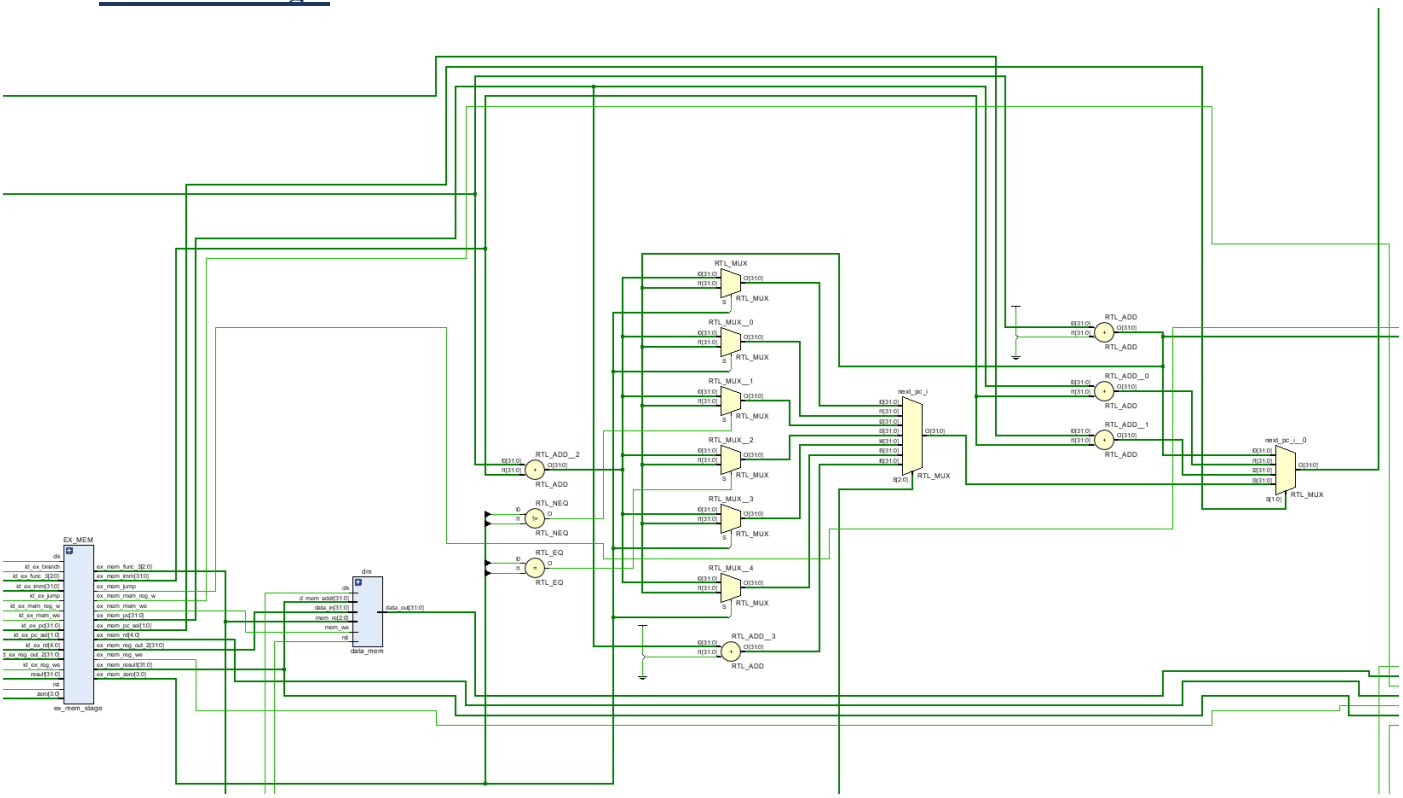
a. IF-ID Stage:

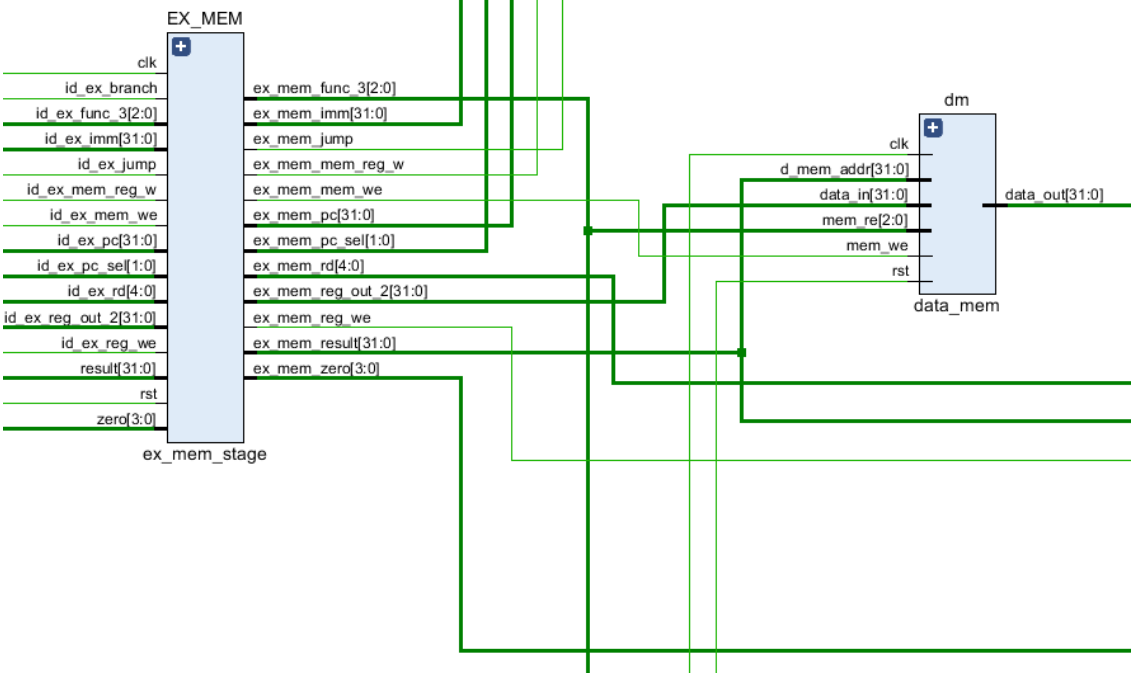


b. ID-EX Stage:

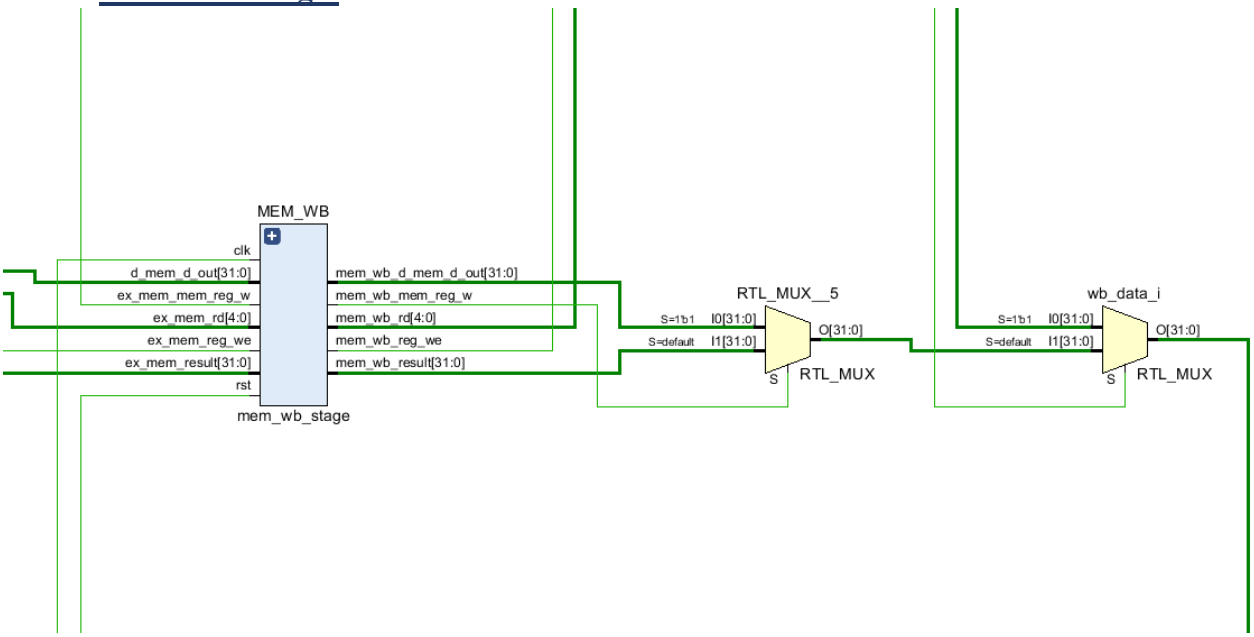


c. EX-MEM Stage:

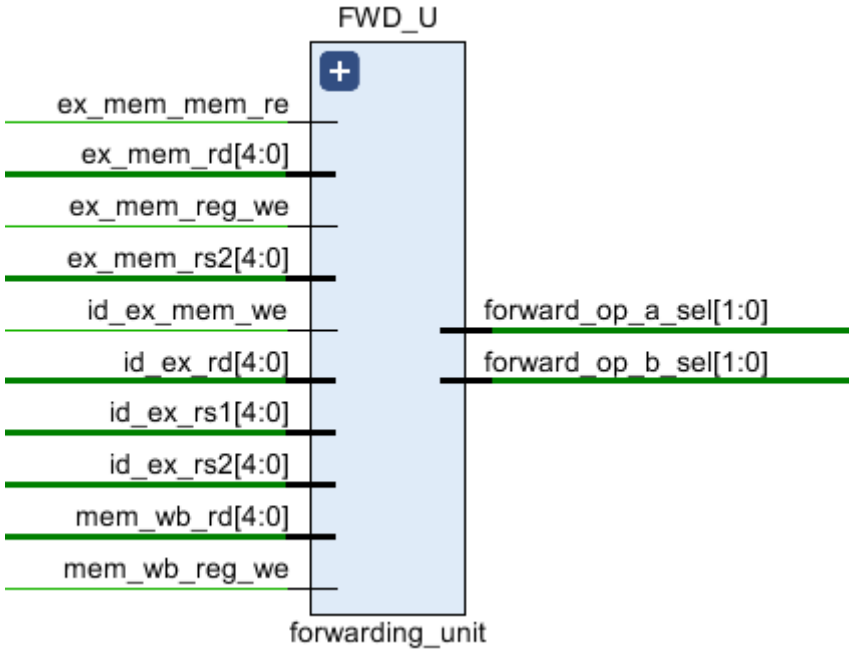


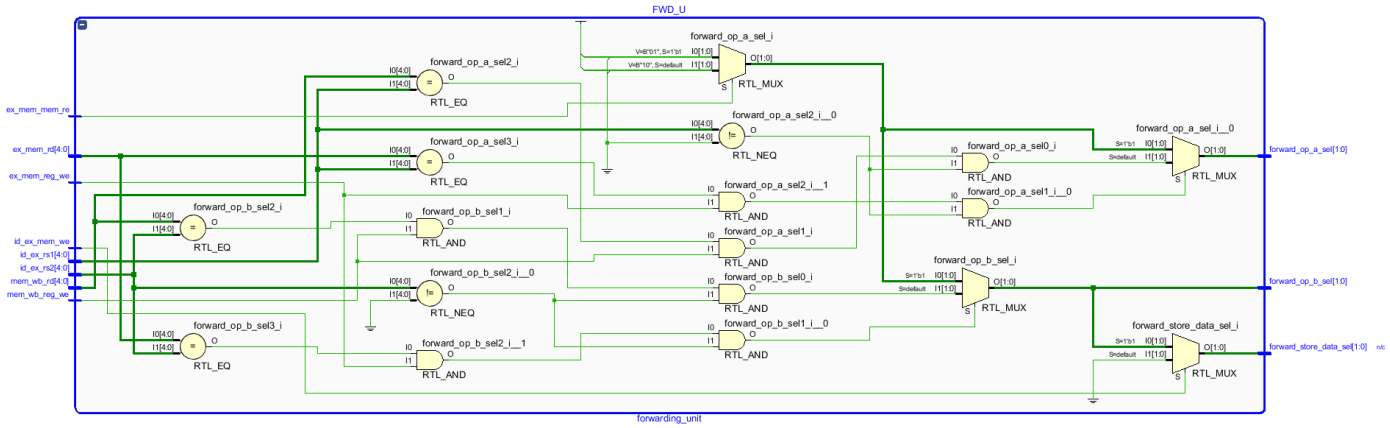


d. MEM-WB Stage:

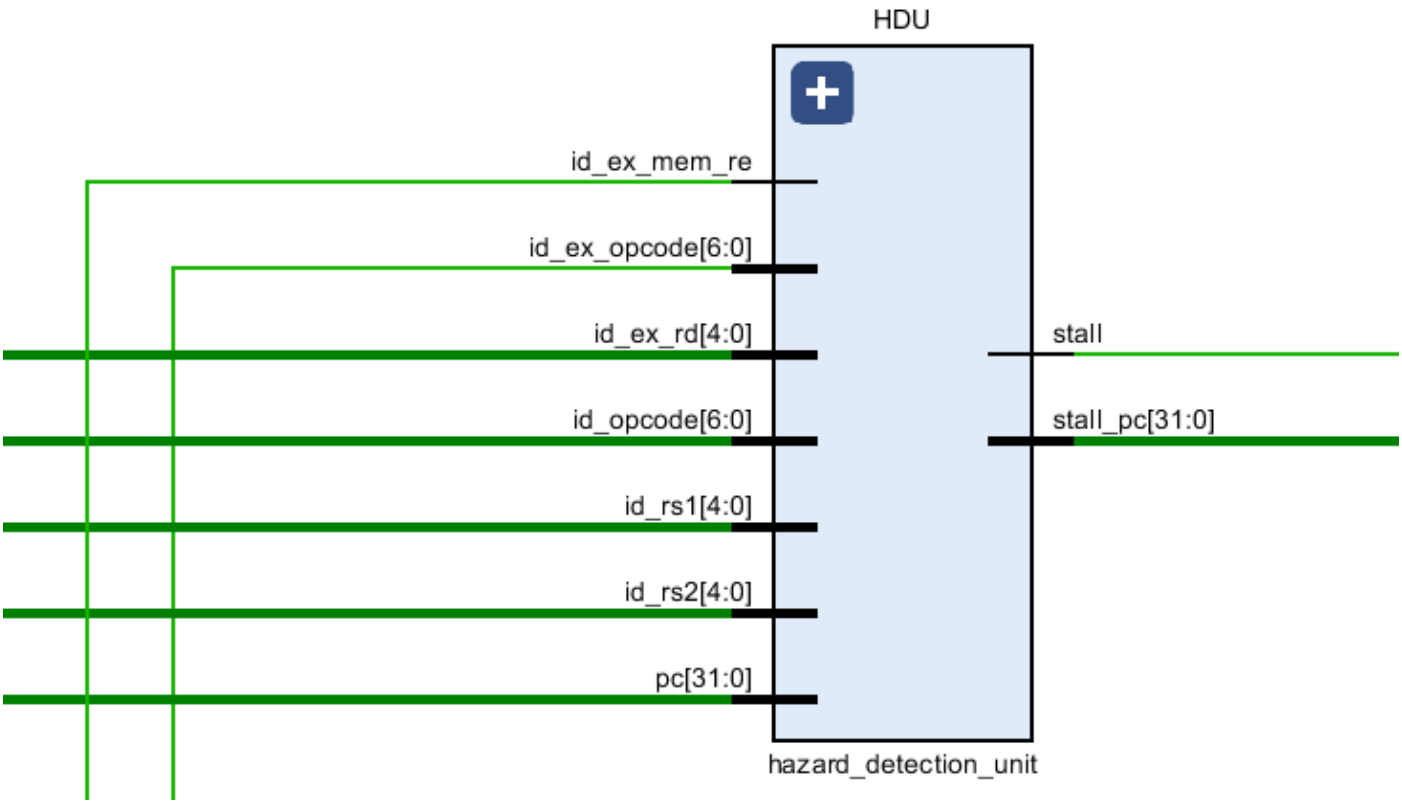


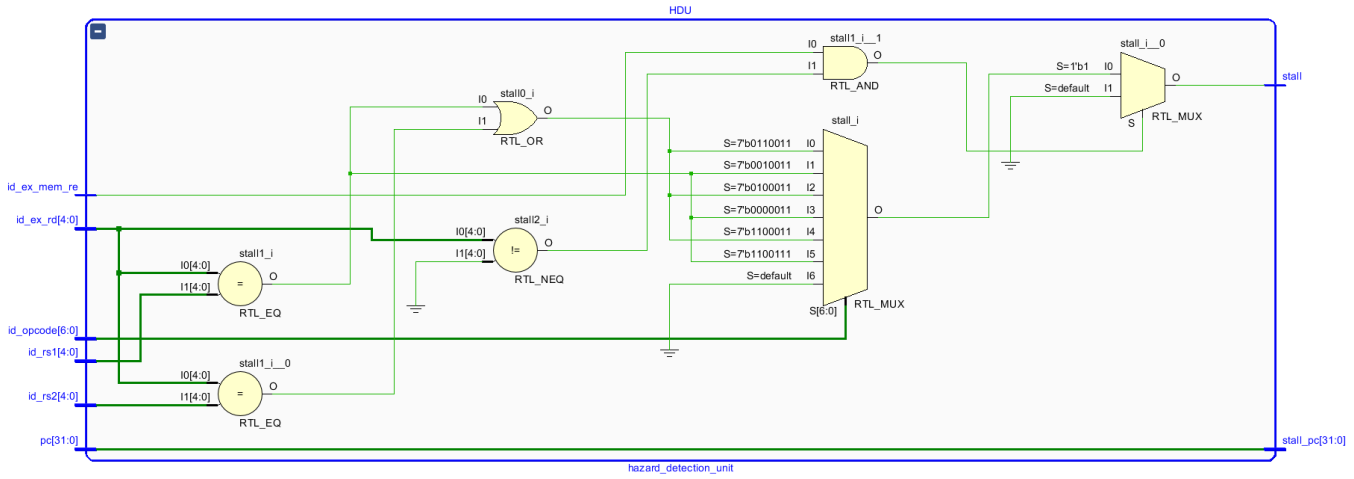
e. Forwarding Unit:



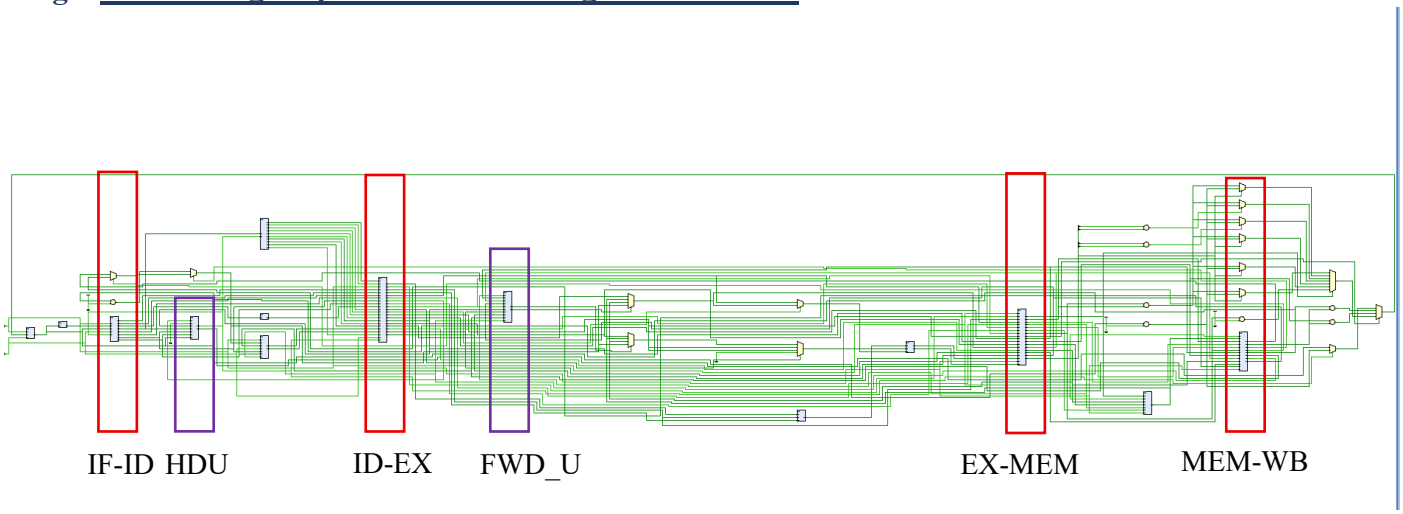


f. Hazard Detection Unit:





g. TOP-5-Stage-Pipeline with handling data Hazards:



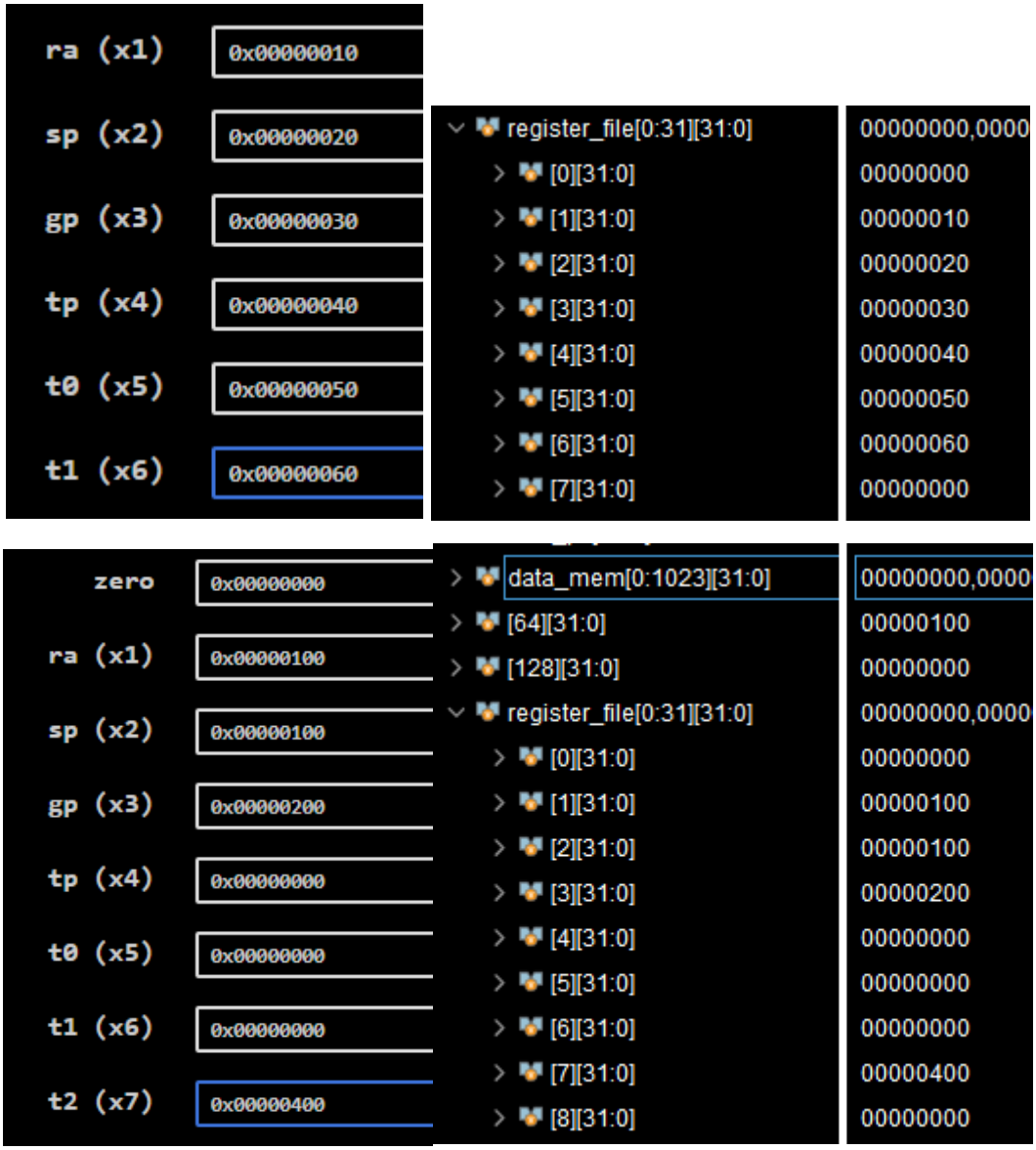
ii. Simulation:

Venus Code:

```
addi x1, x0, 0x10
add x2, x1, x1
add x3, x1, x2
add x4, x1, x3
add x5, x1, x4
add x6, x1, x5
```

```
addi x1, x0, 0x100
sw x1, 0(x1)
lw x2, 0(x1)
addi x3, x2, 0x100      # x3 = 100
add x7, x3, x3        # x7 = 7
```

Screenshots:



These are results comparison. You will observe that the results are incredibly true.

2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, I implemented and analyzed **data hazard handling** in a 5-stage pipelined RISC-V processor. I explored how forwarding paths resolve most **ALU-ALU dependencies** by directly supplying results from later pipeline stages back to the EX-stage, thus avoiding unnecessary stalls. I also identified the special case of **load-use hazards**, where the loaded value is not available until the MEM stage, requiring a one-cycle stall before the dependent instruction can execute correctly. Through practical implementation, I confirmed that forwarding improves pipeline efficiency by minimizing stalls, while hazard detection logic ensures correctness when stalls are unavoidable. This experiment reinforced the importance of balancing performance and correctness in pipelined processor design.